

Proscalar External Ingress API Specification

Proscalar External Ingress API Specification: Kong Gateway Integration

This document is the official API integration guide for external vendors, engineering teams, and system integrators sending telemetry and alarm webhooks from **Proscalar Cloud / Access Control Hardware** into the **iMOPS Common Data Environment (CDE)** through **Kong Gateway**.

1. Gateway Connection Details

All external webhook calls must target the perimeter **Kong Gateway** ingress. Direct connections to internal backend services are blocked by firewall rules.

Parameter	Development / Local	Production Environment
Protocol	HTTP	HTTPS (SSL/TLS 1.2+)
Host / IP	localhost (or server LAN IP)	api.imops.yourdomain.com
Port	8088 (HTTP)	8443 (HTTPS)
Endpoint Path	/api/proscalar/webhook	/api/proscalar/webhook
HTTP Method	POST	POST
Content-Type	application/json	application/json

2. Authentication & Security Guardrails

2.1 HTTP Basic Authentication

Kong enforces strict perimeter authentication. Every request **MUST** supply an `Authorization` header with valid Basic credentials:

- **Username:** proscalar@gmail.com
- **Password:** fwpykjcf2i8fcoP05yEz (or environment vault secret)

Authorization Header Format:

Authorization: Basic cHJvc2NhbGFyQGdtYWlsLmNvbTpmZ3BreWpjZjJpOGZjb1AwNXlFeg==

NOTE

Requests without valid credentials or malformed authorization headers are immediately rejected at the network edge with **HTTP 401 Unauthorized**.

2.2 Edge Protection Policies

Kong automatically enforces the following perimeter policies:

1. **Rate Limiting:** Maximum **120 requests per minute** per client. Bursts exceeding this threshold receive **HTTP 429 Too Many Requests**.
2. **Payload Size Limit:** Maximum payload body size is **5 MB** (HTTP 413 Payload Too Large on overflow).
3. **Correlation Tracking:** Kong attaches a unique **X-Request-ID** (UUIDv4) header to both request and response for end-to-end distributed tracing.

3. Webhook Event Envelope Formats

Proscalar transmits events in two standard envelope schemas:

Type A: Point / Device Event (`type: "event.publish"`)

Used for single-device hardware triggers, access events, and point tampers:

```
{
  "type": "event.publish",
  "timestamp": "2026-10-07T05:00:00.000Z",
  "data": {
    "id": "unique-event-guid",
    "eventTimestamp": "2026-10-07T04:59:58.000Z",
    "eventCode": "5002",
    "eventSourceId": "panel-or-controller-id",
    "eventSourceName": "OT-CTR-MAIN GATE",
    "eventPointId": "reader-or-sensor-id",
    "eventPointName": "OT-MAIN GATE",
    "message": "Logical Door Forced Open ACTIVE.",
    "receivedTimestamp": "2026-10-07T04:59:59Z"
  }
}
```

Type B: Multi-Zone Alarm Group (`type: "alarm.publish"`)

Used for multi-zone clusters, aggregated fire signals, and zone partition alerts:

```
{
```

```

"type": "alarm.publish",
"timestamp": "2026-10-07T05:15:00.000Z",
"data": {
  "alarmGroupId": "cluster-group-id-99",
  "groupEventCode": "12008",
  "name": "FIRE SIGNAL ALARM GROUP",
  "msg": "Alarm Monitoring Group Alarm. Alarm Zone OT-OPEN AREA",
  "eventSourceId": "panel-source-id",
  "eventSourceName": "OT-CTR-D00R 1",
  "eventTimestamp": "2026-10-07T05:14:58.000Z",
  "accessZones": [
    {
      "alarmZoneId": "zone-guid-1",
      "name": "ZONE A",
      "type": "FIRE",
      "state": "TRIGGERED",
      "eventCode": "12000"
    }
  ]
}
}

```

4. The 6 Standard Webhook Calls

The following 6 calls represent the complete certified integration catalog matching `docs/imops_proscalar.json`.

6 SUPPORTED VENDOR WEBHOOK CALLS

- ↳ Call 1: Door Forced Open (Code 5002) -> High Active Incident
- ↳ Call 2: Reader Tamper ACTIVE (Code 4001) -> High Active Incident
- ↳ Call 3: Reader Tamper IDLE (Code 4000) -> Auto-Resolves Call 2
- ↳ Call 4: Fire Alarm Group (Code 12008)-> CRITICAL Safety Override
- ↳ Call 5: AC Power Failure (Code 1000) -> Hardware Failure Alarm
- ↳ Call 6: Emergency Door Release (Code 5014) -> CRITICAL Life-Safety Alert

Call 1: Door Forced Open (Active Access Alarm)

- **Purpose:** Triggered when a physical door contact is broken without a valid badge swipe.
- **Event Code:** 5002
- **Severity in iMOPS:** HIGH
- **Dashboard Tab:** ATTEND NOW (Active incident with SOP checklist)

Payload:

```
{
  "type": "event.publish",
  "timestamp": "2026-06-09T03:00:00.000Z",
  "data": {
    "id": "mock-access-event-id",
    "eventTimestamp": "2026-06-09T02:59:58.000Z",
    "eventCode": "5002",
    "eventSourceId": "proscalar-test-source-id",
    "eventSourceName": "OT-CTR-MAIN GATE",
    "eventPointId": "proscalar-test-point-id",
    "eventPointName": "OT-MAIN GATE",
    "message": "Logical Door Forced Open ACTIVE.",
    "receivedTimestamp": "2026-06-09T02:59:59Z"
  }
}
```

Code Snippets:

• cURL (Linux / macOS):

```
curl -i -X POST http://localhost:8088/api/proscalar/webhook \\\
-u "proscalar@gmail.com:fwpyjcf2i8fcoP05yEz" \\\
-H "Content-Type: application/json" \\\
-d '{"type":"event.publish","timestamp":"2026-06-09T03:00:00.000Z","data":{"id":"mock-access-event-id","eventTimestamp":"2026-06-09T02:59:58.000Z","eventCode":"5002","eventSourceId":"proscalar-test-source-id","eventSourceName":"OT-CTR-MAIN GATE","eventPointId":"proscalar-test-point-id","eventPointName":"OT-MAIN GATE","message":"Logical Door Forced Open ACTIVE."}}'
```

• Windows PowerShell:

```
$headers = @{ Authorization = "Basic " +
    [Convert]::ToBase64String([Text.Encoding]::ASCII.GetBytes("proscalar@gmail.com:fwpyjcf2i8fcoP05yEz")) }

$body = @'
```

```
{ "type": "event.publish", "timestamp": "2026-06-09T03:00:00.000Z", "data": { "id": "mock-access-event-id", "eventTimestamp": "2026-06-09T02:59:58.000Z", "eventCode": "5002", "eventSourceId": "proscalar-test-source-id", "eventSourceName": "OT-CTR-MAIN GATE", "eventPointId": "proscalar-test-point-id", "eventPointName": "OT-MAIN GATE", "message": "Logical Door Forced Open ACTIVE." } }
```

'@

```
Invoke-RestMethod -Uri "http://localhost:8088/api/proscalar/webhook" -Method Post -Headers $headers  
-ContentType "application/json" -Body $body
```

• Python (requests):

```
import requests  
  
from requests.auth import HTTPBasicAuth  
  
url = "http://localhost:8088/api/proscalar/webhook"  
auth = HTTPBasicAuth("proscalar@gmail.com", "fwpyjcf2i8fcoP05yEz")  
payload = {  
    "type": "event.publish",  
    "timestamp": "2026-06-09T03:00:00.000Z",  
    "data": {  
        "id": "mock-access-event-id",  
        "eventTimestamp": "2026-06-09T02:59:58.000Z",  
        "eventCode": "5002",  
        "eventSourceId": "proscalar-test-source-id",  
        "eventSourceName": "OT-CTR-MAIN GATE",  
        "eventPointId": "proscalar-test-point-id",  
        "eventPointName": "OT-MAIN GATE",  
        "message": "Logical Door Forced Open ACTIVE."  
    }  
}  
  
response = requests.post(url, json=payload, auth=auth)  
print(response.status_code, response.json())
```

Call 2: Reader Tamper (Active Tamper Alarm)

- **Purpose:** Triggered when a physical badge reader is unscrewed or removed from the wall.
- **Event Code:** 4001
- **Severity in iMOPS:** HIGH
- **Dashboard Tab:** ATTEND NOW (Active incident created on device CV485)

Payload:

```
{
  "type": "event.publish",
  "timestamp": "2026-06-09T03:05:00.000Z",
  "data": {
    "id": "mock-tamper-event-id",
    "eventTimestamp": "2026-06-09T03:04:58.000Z",
    "eventCode": "4001",
    "eventSourceId": "proscalar-test-source-id",
    "eventSourceName": "178",
    "eventPointId": "proscalar-test-point-id",
    "eventPointName": "CV485",
    "message": "Logical Reader Tamper ACTIVE.",
    "receivedTimestamp": "2026-06-09T03:04:59Z"
  }
}
```

Code Snippets:

• Windows PowerShell:

```
$body = @'

{"type":"event.publish","timestamp":"2026-06-09T03:05:00.000Z","data":{"id":"mock-tamper-event-id","eventTimestamp":"2026-06-09T03:04:58.000Z","eventCode":"4001","eventSourceId":"proscalar-test-source-id","eventSourceName":"178","eventPointId":"proscalar-test-point-id","eventPointName":"CV485","message":"Logical Reader Tamper ACTIVE."}}

'@

Invoke-RestMethod -Uri "http://localhost:8088/api/proscalar/webhook" -Method Post -Headers $headers -ContentType "application/json" -Body $body
```

Call 3: Reader Tamper Idle (Auto-Resolution Event)

- **Purpose:** Triggered when the physical reader is re-attached and tamper switch is closed.
- **Event Code:** 4000 (Paired restoration code for 4001)
- **Severity in iMOPS:** LOW / Auto-Resolve
- **Dashboard Tab:** PAST INCIDENTS

IMPORTANT

Why no new alarm card appears on dashboard:

Code 4000 is an **auto-resolution restoration code**. Its purpose is to search for any open incident opened by 4001 on device CV485 and mark it **Resolved** in real time via Socket.IO. It does not spawn a new alarm.

Payload:

```
{
  "type": "event.publish",
  "timestamp": "2026-06-09T03:10:00.000Z",
  "data": {
    "id": "mock-tamper-idle-id",
    "eventTimestamp": "2026-06-09T03:09:58.000Z",
    "eventCode": "4000",
    "eventSourceId": "proscalar-test-source-id",
    "eventSourceName": "178",
    "eventPointId": "proscalar-test-point-id",
    "eventPointName": "CV485",
    "message": "Logical Reader Tamper IDLE.",
    "receivedTimestamp": "2026-06-09T03:09:59Z"
  }
}
```

Code Snippets:

• Windows PowerShell:

```
$body = @'
{"type":"event.publish","timestamp":"2026-06-09T03:10:00.000Z","data":{"id":"mock-tamper-idle-id","eventTimestamp":"2026-06-09T03:09:58.000Z","eventCode":"4000","eventSourceId":"proscalar-test-source-id","eventSourceName":"178","eventPointId":"proscalar-test-point-id","eventPointName":"CV485","message":"Logical Reader Tamper IDLE."}}
'@

Invoke-RestMethod -Uri "http://localhost:8088/api/proscalar/webhook" -Method Post -Headers $headers
-ContentType "application/json" -Body $body
```

Call 4: Fire Alarm Group (Multi-Zone Safety Aggregation)

- **Purpose:** Triggered when an entire partition group or multi-zone fire cluster is activated.
- **Event Code:** 12008 (with `accessZones[].type == "FIRE"`)

- **Severity in iMOPS:** **CRITICAL** (Dark Red highlight row with immediate audio siren)
- **Dashboard Tab:** **ATTEND NOW**

Payload:

```
{
  "type": "alarm.publish",
  "timestamp": "2026-06-09T03:15:00.000Z",
  "data": {
    "alarmGroupId": "mock-alarm-group-id-99",
    "groupEventCode": "12008",
    "name": "FIRE SIGNAL ALARM GROUP",
    "msg": "Alarm Monitoring Group Alarm. Alarm Zone OT-OPEN AREA",
    "eventSourceId": "proscalar-test-source-id",
    "eventSourceName": "OT-CTR-D00R 1",
    "eventTimestamp": "2026-06-09T03:14:58.000Z",
    "accessZones": [
      {
        "alarmZoneId": "mock-zone-id-a",
        "name": "ZONE A",
        "type": "FIRE",
        "state": "TRIGGERED",
        "eventCode": "12000"
      },
      {
        "alarmZoneId": "mock-zone-id-b",
        "name": "ZONE B",
        "type": "INSTANT",
        "state": "TRIGGERED",
        "eventCode": "12000"
      }
    ]
  }
}
```

Code Snippets:

- **Windows PowerShell:**

```
$body = @'
```

```
{
  "type": "alarm.publish",
  "timestamp": "2026-06-09T03:15:00.000Z",
  "data": {
    "alarmGroupId": "mock-alarm-group-id-99",
    "groupEventCode": "12008",
    "name": "FIRE SIGNAL ALARM GROUP",
    "msg": "Alarm Monitoring Group Alarm. Alarm Zone OT-OPEN AREA",
    "eventSourceId": "proscalar-test-source-id",
    "eventSourceName": "OT-CTR-D00R",
    "eventTimestamp": "2026-06-09T03:14:58.000Z",
    "accessZones": [
      {
        "alarmZoneId": "mock-zone-id-a",
        "name": "ZONE A",
        "type": "FIRE",
        "state": "TRIGGERED",
        "eventCode": "12000"
      },
      {
        "alarmZoneId": "mock-zone-id-b",
        "name": "ZONE B",
        "type": "INSTANT",
        "state": "TRIGGERED",
        "eventCode": "12000"
      }
    ]
  }
}
```

'@

```
Invoke-RestMethod -Uri "http://localhost:8088/api/proscalar/webhook" -Method Post -Headers $headers
-ContentType "application/json" -Body $body
```

Call 5: System & Hardware Alarm: AC Power Failure

- **Purpose:** Triggered when building mains power is cut and the controller switches to backup battery.
- **Event Code:** 1000
- **Severity in iMOPS:** **HIGH**
- **Dashboard Tab:** **ATTEND NOW** (Power Failure - Main Enclosure Power)

Payload:

```
{
  "type": "event.publish",
  "timestamp": "2026-06-09T03:20:00.000Z",
  "data": {
    "id": "mock-power-failure-id",
    "eventTimestamp": "2026-06-09T03:19:58.000Z",
    "eventCode": "1000",
    "eventSourceId": "proscalar-test-source-id",
    "eventSourceName": "OT-CTR-MAIN GATE",
    "eventPointId": "proscalar-test-point-id",
    "eventPointName": "Main Enclosure Power",
    "message": "Controller AC Power Failure ACTIVE.",
    "receivedTimestamp": "2026-06-09T03:19:59Z"
  }
}
```

Code Snippets:

- **Windows PowerShell:**

```
$body = '@'

{"type": "event.publish", "timestamp": "2026-06-09T03:20:00.000Z", "data": {
  "id": "mock-power-failure-id",
  "eventTimestamp": "2026-06-09T03:19:58.000Z",
  "eventCode": "1000",
  "eventSourceId": "proscalar-test-source-id",
  "eventSourceName": "OT-CTR-MAIN GATE",
  "eventPointId": "proscalar-test-point-id",
  "eventPointName": "Main Enclosure Power",
  "message": "Controller AC Power Failure ACTIVE."
}}
```

```
'@
```

```
Invoke-RestMethod -Uri "http://localhost:8088/api/proscalar/webhook" -Method Post -Headers $headers  
-ContentType "application/json" -Body $body
```

Call 6: Emergency Door Release (Life Safety Button)

- **Purpose:** Triggered when an occupant smashes a green Emergency Door Release (break-glass / push-to-exit) button.
- **Event Code:** 5014
- **Severity in iMOPS:** **CRITICAL**
- **Dashboard Tab:** **ATTEND NOW** (Emergency Exit Opened - EMERGENCY DOOR RELEASE BUTTON)

Payload:

```
{  
  "type": "event.publish",  
  "timestamp": "2026-06-09T03:25:00.000Z",  
  "data": {  
    "id": "mock-emergency-release-id",  
    "eventTimestamp": "2026-06-09T03:24:58.000Z",  
    "eventCode": "5014",  
    "eventSourceId": "proscalar-test-source-id",  
    "eventSourceName": "OT-CTR-MAIN GATE",  
    "eventPointId": "proscalar-test-point-id",  
    "eventPointName": "EMERGENCY DOOR RELEASE BUTTON",  
    "message": "Emergency Exit Button Pressed ACTIVE.",  
    "receivedTimestamp": "2026-06-09T03:24:59Z"  
  }  
}
```

Code Snippets:

- **Windows PowerShell:**

```
$body = '@  
  
{  
  "type": "event.publish", "timestamp": "2026-06-09T03:25:00.000Z", "data": {  
    "id": "mock-emergency-release-id", "eventTimestamp": "2026-06-09T03:24:58.000Z", "eventCode": "5014", "eventSourceId": "proscalar-test-source-id", "eventSourceName": "OT-CTR-MAIN GATE", "eventPointId": "proscalar-test-point-id", "eventPointName": "EMERGENCY DOOR RELEASE BUTTON", "message": "Emergency Exit Button Pressed ACTIVE." } }  
  
'@
```

```
Invoke-RestMethod -Uri "http://localhost:8088/api/proscalar/webhook" -Method Post -Headers $headers
-ContentType "application/json" -Body $body
```

5. Expected Server Responses

5.1 Success Response (200 OK)

When Kong authenticates the caller and the workflow engine processes the payload, the client receives:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Server: kong/3.9.3
X-Request-ID: 9ed73eac-8198-4126-95b3-7076dd332cf5
RateLimit-Remaining: 119
RateLimit-Limit: 120
```

```
{
  "success": true,
  "status": "processed",
  "pipeline": "Option C (Kong Gateway + n8n)",
  "timestamp": "2026-10-07T04:18:07.747-04:00",
  "executionId": "635"
}
```

5.2 Error Response Codes

| HTTP Status | Reason | What it means | How to fix |

| :--- | :--- | :--- | :--- |

| **401 Unauthorized** | Authentication Failure | Missing or incorrect Basic Auth credentials | Supply Authorization: Basic ... header with registered credentials. |

| **404 Not Found** | Incorrect Route | Request targeted an undefined path | Target POST /api/proscalar/webhook on proxy port 8088 (or 8443). |

| **413 Payload Too Large** | Size Limit Exceeded | Payload size > 5 MB | Compress or trim metadata arrays before transmitting. |

| **422 Unprocessable Entity** | Malformed JSON | Body is not valid JSON or quotes were corrupted | Ensure valid JSON formatting without raw trailing commas or broken escape sequences. |

| **429 Too Many Requests** | Rate Limit Exceeded | Rate exceeded 120 requests in 1 minute | Back off transmission rate until RateLimit-Reset seconds elapse. |

| **502 Bad Gateway** | Upstream Down | Kong cannot connect to n8n or backend container | Check container health via docker ps. |

6. How to Test the Calls (1-Click Runbook)

Method 1: Windows 1-Click Batch Runner (.bat)

Run the pre-configured script from Command Prompt or double-click in File Explorer:

```
# Run all 6 calls sequentially:
c:\Projects\cde\apigw\scripts\test_imops_proscalar_collection.bat all

# Run only scenario 1 (Door Forced):
c:\Projects\cde\apigw\scripts\test_imops_proscalar_collection.bat 1

# Run only scenario 3 (Auto-Resolve Tamper):
c:\Projects\cde\apigw\scripts\test_imops_proscalar_collection.bat 3
```

Method 2: Windows PowerShell Runner (.ps1)

```
powershell -ExecutionPolicy Bypass -File c:\
\Projects\cde\apigw\scripts\test_imops_proscalar_collection.ps1 all
```

Method 3: Postman Collection Import

1. In Postman, click **Import** → select docs/imops_proscalar.json.
2. Set variables:
 - BACKEND_API = http://localhost:8088
 - PROSCALAR_USER = proscalar@gmail.com
 - PROSCALAR_PASS = fwpkyjcf2i8fcoP05yEz
3. Click **Send** on any request.

7. Verification Dashboards

After firing any webhook, verify the results live:

1. iMOPS Operator Dashboard:

- Open the dashboard in browser.
- View the **ATTEND NOW** table for active incidents (Door Forced Open, Power Failure, Emergency Exit, Fire Alarm).
- View the **PAST INCIDENTS** table for auto-resolved incidents (Reader Tamper).

2. n8n Workflow Executions:

- Open <https://localhost:5678> → Click **Executions** in sidebar to view visual node-by-node execution graphs.

3. Kong Manager Dashboard:

- Open **`http://localhost:8002`** → Click **Gateway Services** → `imops-proscalar-n8n-service` to inspect real-time throughput metrics.